TOPIC 1: INTRODUCTION

First Palindromic String in Array

Aim:

To find the first palindromic string in the given array.
To return empty string if no palindrome exists

Algorithm:

- Start the program.
- Traverse each string in the array.
- Reverse the string.
- Compare with original string.
- Return first matching palindrome else "".

Code:

```python
def firstPalindrome(words):
    for word in words:
        if word == word[::-1]:
            return word
    return ""
n = int(input("Enter number of words: "))
words = []

for i in range(n):
    word = input(f"Enter word {i+1}: ")
    words.append(word)
result = firstPalindrome(words)
print("First Palindromic String:", result)
```

Input:

```
Enter number of words: 3
Enter word 1: abc
Enter word 2: car
Enter word 3: ada
```

Output:

```
First Palindromic String: ada
```

Result:

Thus, the first palindromic string is found successfully.

2. Count Common Elements Between Two Arrays

Aim:

To count elements of nums1 present in nums2.
To count elements of nums2 present in nums1.

Algorithm:

- Start the program.
- Convert both arrays into sets.
- Count matching elements from nums1 in nums2.
- Count matching elements from nums2 in nums1.
- Return [answer1, answer2].

Code:

```python
def findCounts(nums1, nums2):
    return [sum(x in nums2 for x in nums1),
            sum(x in nums1 for x in nums2)]

# Example
nums1 = [1, 2, 3]
nums2 = [2, 3, 4]

print(findCounts(nums1, nums2))
```

Input:

```
nums1 = [1, 2, 3]
nums2 = [2, 3, 4]
```

Output:

```
PS C:\Users\Anandpaul\OneDrive\Desktop\DAA_Lab> python p2.py
[2, 2]
```

Result:

Thus, common element counts are calculated.

3. Sum of Squares of Distinct Counts of Subarrays

Aim:

To compute distinct count of each subarray.
To return sum of squares of distinct counts.

Algorithm:

- Start the program.
- Generate all possible subarrays.
- Count distinct elements in each subarray.
- Square the distinct count.
- Add to total sum**.**

Code:

```
p3.py > ...
def sumOfSquares(nums):
    n = len(nums)
    total = 0

    for i in range(n):
        dist (variable) j: int
        for j in range(i, n):
            distinct.add(nums[j])
            total += len(distinct) ** 2

    return total


# Taking input from user
n = int(input("Enter size of array: "))
nums = list(map(int, input("Enter elements: ").split()))

# Function call
result = sumOfSquares(nums)

# Output
print("Sum of squares of distinct counts:", result)
```

Input:

```
Enter size of array: 3
Enter elements: 1 2 1
```

Output:

```
Sum of squares of distinct counts: 15
```

Result:

Thus, the required sum is calculated successfully.

4. Count Equal Pairs Divisible by k

Aim:

To count pairs where nums[i] = nums[j].
To ensure (i × j) is divisible by k.

Algorithm:

- Start the program.
- Use nested loop to generate pairs.
- Check equality condition.
- Check divisibility by k.
- Count valid pairs.

Code:

```python
def countPairs(nums, k):
    n = len(nums)
    count = 0

    for i in range(n):
        for j in range(i + 1, n):
            if nums[i] == nums[j] and (i * j) % k == 0:
                count += 1

    return count


# Taking input
n = int(input("Enter size of array: "))
nums = list(map(int, input("Enter elements: ").split()))
k = int(input("Enter value of k: "))

# Function call
result = countPairs(nums, k)

# Output
print("Number of valid pairs:", result)
```

Input:

```
Enter size of array: 5
Enter elements: 1 2 3 2 2
Enter value of k: 2
```

Output:

```
Number of valid pairs: 2
```

Result:

Thus, valid pairs are counted correctly.

5. Find Maximum Element (Least Time Complexity O(n))

Aim:

To find maximum element in array.
To achieve minimum time complexity.

Algorithm:

- Start the program.
- Assume first element as max.
- Traverse remaining elements.
- Update max if larger element found.
- Print max value.

Code:

p5.py > ...

```python
def findMaximum(nums):
    return max(nums)


# Taking input from user
n = int(input("Enter size of array: "))
nums = list(map(int, input("Enter elements: ").split()))

# Function call
result = findMaximum(nums)

# Output
print("Maximum element:", result)
```

Input:

```
Enter size of array: 5
Enter elements: 1 2 3 4 5
```

Output:

```
Maximum element: 5
```

Result:

Thus, maximum element is found efficiently.

6. Sort List and Find Maximum

Aim:

To sort the list using efficient sorting.
To return maximum element from sorted list.

Algorithm:

- Start the program.
- If list empty return None.
- Sort list using merge sort.
- Take last element as maximum.
- Print result.

Code:

```python
def find_max_after_sort(numbers):
    if not numbers:
        return None
    numbers.sort()
    return numbers[-1]

numbers = eval(input())
result = find_max_after_sort(numbers)
print(result)
```

Input:

```
PS C:\Users\Anandpaul\OneDrive\Desktop\DAA_Lab> python p6.py
[3,3,2,4,3]
```

Output:

```
4
```

Result:

Thus, sorted list and maximum element are obtained.

7. Unique Elements in List

Aim:

To remove duplicate elements from list.
To create new list with unique values**.**

Algorithm:

- Start the program.
- Create empty result list.
- Traverse input list.
- Add element if not present.
- Return result list.

Code:

```python
def unique_elements(numbers):
    unique_list = []
    for num in numbers:
        if num not in unique_list:
            unique_list.append(num)
    return unique_list

numbers = eval(input())
result = unique_elements(numbers)
print(result)
```

Input:

```
PS C:\Users\Anandpaul\OneDrive\Desktop\DAA_Lab> python p7.py
[3,2,4,5,6,2,3,4,5]
```

Output:

```
[3, 2, 4, 5, 6]
```

Result:

Thus, unique elements are extracted.

8. Bubble Sort

Aim:

To sort array using bubble sort.
To arrange elements in ascending order.

Algorithm:

- Start the program.
- Compare adjacent elements.
- Swap if necessary.
- Repeat passes n times.
- Return sorted array.

Code:

p8.py > ...

```python
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

numbers = eval(input())
sorted_numbers = bubble_sort(numbers)
print(sorted_numbers)
```

Input:

```
PS C:\Users\Anandpaul\OneDrive\Desktop\DAA_Lab> python p8.py
[5,4,9,2]
```

Output:

```
[2, 4, 5, 9]
```

Result: Thus, array is sorted using bubble sort.

9. Binary Search

Aim:

To search element in sorted array.
To reduce search space using divide and conquer.

Algorithm:

- Start the program.
- Initialize low and high.
- Find mid element.
- Compare with key.
- Repeat until found/not found.

Code:

```python
def binary_search(arr, key):
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1

arr = eval(input())
key = int(input())

result = binary_search(arr, key)

if result != -1:
    print("Element", key, "is found at position", result)
else:
    print("Element", key, "is not found")
```

Input:

```
PS C:\Users\Anandpaul\OneDrive\Desktop\DAA_Lab> python p9.py
[2,3,5,6,7,8,9]
7
```

Output:

```
Element 7 is found at position 4
```

Result: Thus, binary search is successful.

10. Merge Sort (O(n log n))

Aim:

To sort array in ascending order.
To achieve O(n log n) complexity.

Algorithm:

- Divide array into halves.
- Recursively sort halves.
- Merge sorted halves.
- Compare elements while merging.
- Return sorted array.

Code:

p10.py > ...

```python
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr)//2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left,right)

def merge(left,right):
    result=[]
    i=j=0
    while i<len(left) and j<len(right):
        if left[i]<right[j]:
            result.append(left[i]); i+=1
        else:
            result.append(right[j]); j+=1
    result+=left[i:]
    result+=right[j:]
    return result

nums = eval(input())
print(merge_sort(nums))
```

Input:

```
PS C:\Users\Anandpaul\OneDrive\Desktop\DAA_Lab> python p10.py
[4,1,6,2]
```

Output:

```
[1, 2, 4, 6]
```

Result: Thus, array is sorted efficiently.

11.

11. Out of Boundary Paths

Aim:

To count number of ways ball moves outside grid.
To calculate total possible paths in N steps.

Algorithm:

- Start recursion from starting cell.
- If outside grid, count 1.
- If steps exhausted return 0.
- Move in 4 directions.
- Return total count.

Code:

```python
def findPaths(m,n,N,i,j):
    dp=[[0]*n for _ in range(m)]
    dp[i][j]=1
    count=0
    for _ in range(N):
        temp=[[0]*n for _ in range(m)]
        for r in range(m):
            for c in range(n):
                if dp[r][c]>0:
                    for x,y in [(1,0),(-1,0),(0,1),(0,-1)]:
                        nr,nc=r+x,c+y
                        if 0<=nr<m and 0<=nc<n:
                            temp[nr][nc]+=dp[r][c]
                        else:
                            count+=dp[r][c]
        dp=temp
    return count

m,n,N,i,j=map(int,input().split())
print(findPaths(m,n,N,i,j))
```

Input:

```
PS C:\Users\Anandpaul\OneDrive\Desktop\DAA_Lab> python p11.py
2 2 2 0 0
```

Output:

```
6
```

Result: Thus, total boundary paths are found.

12. House Robber II (Circular)

Aim:

To maximize robbed money from circular houses.
To avoid robbing adjacent houses.

Algorithm:

- Handle single house case.
- Solve excluding first house.
- Solve excluding last house.
- Take maximum of both.
- Return result.

Code:

p12.py > ...

```python
def rob_line(nums):
    prev=curr=0
    for num in nums:
        prev,curr=curr,max(curr,prev+num)
    return curr

def rob(nums):
    if len(nums)==1:
        return nums[0]
    return max(rob_line(nums[:-1]),rob_line(nums[1:]))

nums=eval(input())
print(rob(nums))
```

Input:

```
PS C:\Users\Anandpaul\OneDrive\Desktop\DAA_Lab> python p12.py
[2,3,2]
```

Output:

```
3
```

Result: Thus, maximum robbery amount is obtained.

13. Climbing Stairs

Aim:

To count distinct ways to reach top.
To use dynamic programming approach.

Algorithm:

- If n≤2 return n.
- Initialize first two steps.
- Loop from 3 to n.
- Compute current = sum of previous two.
- Return result.

Code:

```python
def climbStairs(n):
    a,b=1,1
    for _ in range(n):
        a,b=b,a+b
    return a

n=int(input())
print(climbStairs(n))
```

Input:

```
PS C:\Users\Anandpaul\OneDrive\Desktop\DAA_Lab> python p13.py
4
```

Output:

```
5
```

Result: Thus, number of ways is calculated.

14. Unique Paths in Grid

Aim:

To find number of unique paths in grid.
To move only right or down.

Algorithm:

- Initialize DP matrix.
- Fill first row and column with 1.
- Fill rest using top + left.
- Continue till end.
- Return bottom-right value.

Code:

```python
def uniquePaths(m,n):
    dp=[1]*n
    for _ in range(m-1):
        for j in range(1,n):
            dp[j]+=dp[j-1]
    return dp[-1]

m,n=map(int,input().split())
print(uniquePaths(m,n))
```

Input:

```
PS C:\Users\Anandpaul\OneDrive\Desktop\DAA_Lab> python p14.py
3 4
```

Output:

```
10
```

Result: Thus, total unique paths are calculated.

15. Large Groups in String

Aim:

To find intervals of large consecutive character groups.
To detect groups with length ≥3.

Algorithm:

- Traverse string.
- Count consecutive characters.
- If count ≥3 store interval.
- Continue till end.
- Return intervals list.

Code:

```
p15.py > ...
1   def largeGroupPositions(s):
2       res=[]
3       i=0
4       for j in range(len(s)):
5           if j==len(s)-1 or s[j]!=s[j+1]:
6               if j-i+1>=3:
7                   res.append([i,j])
8               i=j+1
9       return res
10
11  s=input()
12  print(largeGroupPositions(s))
```

Input:

```
PS C:\Users\Anandpaul\OneDrive\Desktop\DAA_Lab> python p15.py
"abxxxxy"
```

Output:

```
[[3, 6]]
```

Result: Thus, large group intervals are identified**.**

16. Champagne Tower

Aim:

To compute fullness of specific glass in tower.
To distribute excess champagne equally.

Algorithm:

- Initialize DP structure.
- Pour champagne at top.
- Distribute excess equally.
- Continue row by row.
- Return queried glass value.

Code:

p16.py > ...

```python
def champagneTower(poured,row,glass):
    dp=[[0]*101 for _ in range(101)]
    dp[0][0]=poured
    for r in range(100):
        for c in range(r+1):
            overflow=max(0,dp[r][c]-1)/2
            dp[r+1][c]+=overflow
            dp[r+1][c+1]+=overflow
    return min(1,dp[row][glass])

poured,row,glass=map(int,input().split())
print(champagneTower(poured,row,glass))
```

Input:

```
PS C:\Users\Anandpaul\OneDrive\Desktop\DAA_Lab> python p16.py
2 1 1
```

Output:

```
0.5
```

Result: Thus, required glass fullness is calculated successfully.